

SML Project - Red Wine Quality Prediction

IMPORTING REQUIRED LIBRARIES

```
In [1]: ► import numpy as np
import matplotlib.pyplot as plt
import pandas as pd
import seaborn as sns
from warnings import filterwarnings
filterwarnings(action='ignore')
```

LOADING DATASET OF RED WINE

```
In [2]: ► wine = pd.read_csv(r'C:\Users\adizr\Downloads\winequality-red.csv')
```

DATA PRE-PROCESSING

```
In [3]: ► wine.sample(25) # Random 25 samples from the dataset
```

Out[3]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
110	7.8	0.560	0.19	1.8	0.104	12.0	47.0	0.99640	3.19	0.93	9.5	5
724	7.5	1.115	0.10	3.1	0.086	5.0	12.0	0.99580	3.54	0.60	11.2	4
514	10.5	0.510	0.64	2.4	0.107	6.0	15.0	0.99730	3.09	0.66	11.8	7
1021	11.3	0.360	0.66	2.4	0.123	3.0	8.0	0.99642	3.20	0.53	11.9	6
1544	8.4	0.370	0.43	2.3	0.063	12.0	19.0	0.99550	3.17	0.81	11.2	7
1385	8.0	0.810	0.25	3.4	0.076	34.0	85.0	0.99668	3.19	0.42	9.2	5
108	8.0	0.330	0.53	2.5	0.091	18.0	80.0	0.99760	3.37	0.80	9.6	6
328	13.4	0.270	0.62	2.6	0.082	6.0	21.0	1.00020	3.16	0.67	9.7	6
1489	6.2	0.570	0.10	2.1	0.048	4.0	11.0	0.99448	3.44	0.76	10.8	6
313	8.6	0.470	0.30	3.0	0.076	30.0	135.0	0.99760	3.30	0.53	9.4	5
657	12.0	0.500	0.59	1.4	0.073	23.0	42.0	0.99800	2.92	0.68	10.5	7
458	10.4	0.330	0.63	2.8	0.084	5.0	22.0	0.99980	3.26	0.74	11.2	7
1566	6.7	0.160	0.64	2.1	0.059	24.0	52.0	0.99494	3.34	0.71	11.2	6
615	9.6	0.600	0.50	2.3	0.079	28.0	71.0	0.99970	3.50	0.57	9.7	5
93	7.7	0.490	0.26	1.9	0.062	9.0	31.0	0.99660	3.39	0.64	9.6	5
305	10.3	0.530	0.48	2.5	0.063	6.0	25.0	0.99980	3.12	0.59	9.3	6
771	9.4	0.685	0.26	2.4	0.082	23.0	143.0	0.99780	3.28	0.55	9.4	5
1192	7.2	0.250	0.37	2.5	0.063	11.0	41.0	0.99439	3.52	0.80	12.4	7
915	8.6	0.315	0.40	2.2	0.079	3.0	6.0	0.99512	3.27	0.67	11.9	6
969	6.7	0.560	0.09	2.9	0.079	7.0	22.0	0.99669	3.46	0.61	10.2	5
619	11.3	0.370	0.41	2.3	0.088	6.0	16.0	0.99880	3.09	0.80	9.3	5
925	8.6	0.220	0.36	1.9	0.064	53.0	77.0	0.99604	3.47	0.87	11.0	7
877	7.7	0.715	0.01	2.1	0.064	31.0	43.0	0.99371	3.41	0.57	11.8	6
1205	7.2	0.360	0.46	2.1	0.074	24.0	44.0	0.99534	3.40	0.85	11.0	7
614	9.2	0.755	0.18	2.2	0.148	10.0	103.0	0.99690	2.87	1.36	10.2	6

```
In [4]: ► wine.head() # First 5 rows of the dataset
```

Out[4]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5
1	7.8	0.88	0.00	2.6	0.098	25.0	67.0	0.9968	3.20	0.68	9.8	5
2	7.8	0.76	0.04	2.3	0.092	15.0	54.0	0.9970	3.26	0.65	9.8	5
3	11.2	0.28	0.56	1.9	0.075	17.0	60.0	0.9980	3.16	0.58	9.8	6
4	7.4	0.70	0.00	1.9	0.076	11.0	34.0	0.9978	3.51	0.56	9.4	5

In [5]: wine.tail() # Last 5 rows of the dataset

Out[5]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
1594	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58	10.5	5
1595	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76	11.2	6
1596	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75	11.0	6
1597	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71	10.2	5
1598	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.66	11.0	6

In [6]: wine.info() # Information of dataset

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1599 entries, 0 to 1598
Data columns (total 12 columns):
#   Column              Non-Null Count  Dtype
---  -
0   fixed acidity        1599 non-null   float64
1   volatile acidity     1599 non-null   float64
2   citric acid          1599 non-null   float64
3   residual sugar       1599 non-null   float64
4   chlorides            1599 non-null   float64
5   free sulfur dioxide  1599 non-null   float64
6   total sulfur dioxide 1599 non-null   float64
7   density              1599 non-null   float64
8   pH                   1599 non-null   float64
9   sulphates            1599 non-null   float64
10  alcohol              1599 non-null   float64
11  quality              1599 non-null   int64
dtypes: float64(11), int64(1)
memory usage: 150.0 KB
```

In [7]: wine.describe()

Out[7]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	
count	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1599.000000	1
mean	8.319637	0.527821	0.270976	2.538806	0.087467	15.874922	46.467792	0.996747	3.311113	0.658149	
std	1.741096	0.179060	0.194801	1.409928	0.047065	10.460157	32.895324	0.001887	0.154386	0.169507	
min	4.600000	0.120000	0.000000	0.900000	0.012000	1.000000	6.000000	0.990070	2.740000	0.330000	
25%	7.100000	0.390000	0.090000	1.900000	0.070000	7.000000	22.000000	0.995600	3.210000	0.550000	
50%	7.900000	0.520000	0.260000	2.200000	0.079000	14.000000	38.000000	0.996750	3.310000	0.620000	
75%	9.200000	0.640000	0.420000	2.600000	0.090000	21.000000	62.000000	0.997835	3.400000	0.730000	
max	15.900000	1.580000	1.000000	15.500000	0.611000	72.000000	289.000000	1.003690	4.010000	2.000000	

In [8]: wine.isnull() # Checking for 0 values

Out[8]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
0	False	False	False	False	False	False	False	False	False	False	False	False
1	False	False	False	False	False	False	False	False	False	False	False	False
2	False	False	False	False	False	False	False	False	False	False	False	False
3	False	False	False	False	False	False	False	False	False	False	False	False
4	False	False	False	False	False	False	False	False	False	False	False	False
...	...	...	...	...	...	...	...	...	...	...	...	...
1594	False	False	False	False	False	False	False	False	False	False	False	False
1595	False	False	False	False	False	False	False	False	False	False	False	False
1596	False	False	False	False	False	False	False	False	False	False	False	False
1597	False	False	False	False	False	False	False	False	False	False	False	False
1598	False	False	False	False	False	False	False	False	False	False	False	False

1599 rows × 12 columns

```
In [9]: wine.isnull().sum()
```

```
Out[9]: fixed acidity      0
volatile acidity    0
citric acid         0
residual sugar      0
chlorides           0
free sulfur dioxide  0
total sulfur dioxide 0
density             0
pH                  0
sulphates           0
alcohol             0
quality             0
dtype: int64
```

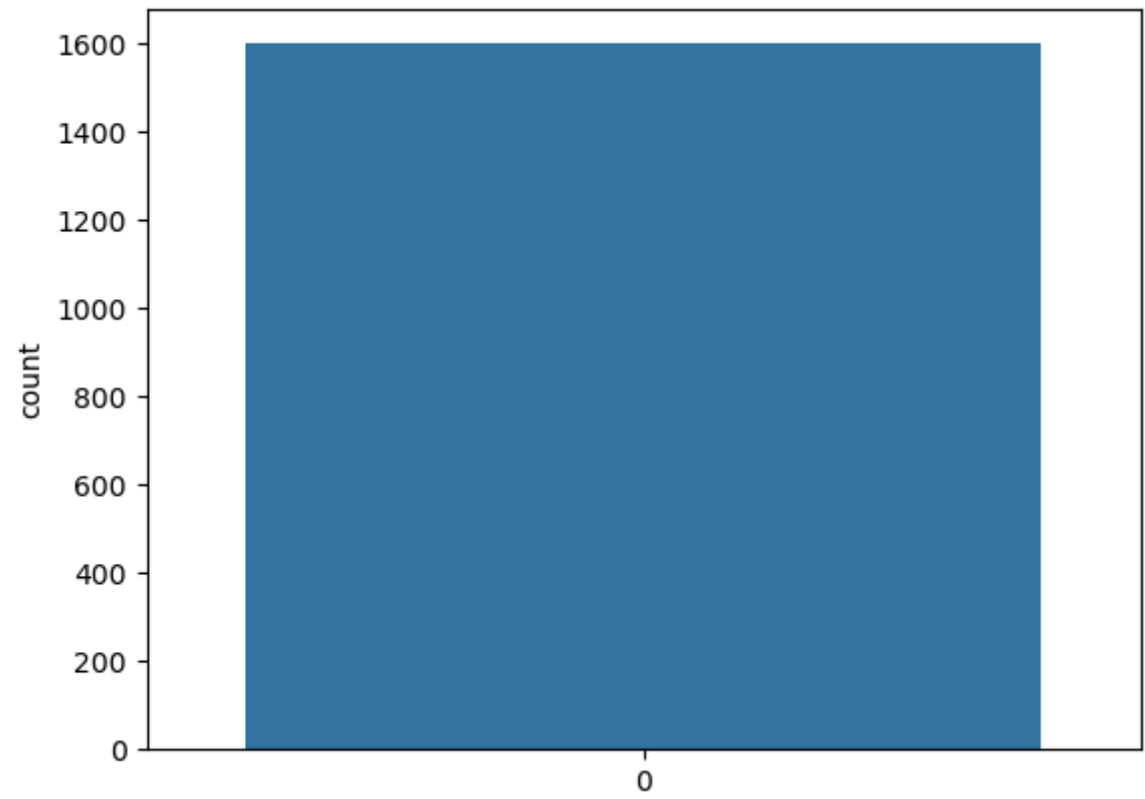
```
In [10]: wine.groupby('quality').mean()
```

Out[10]:

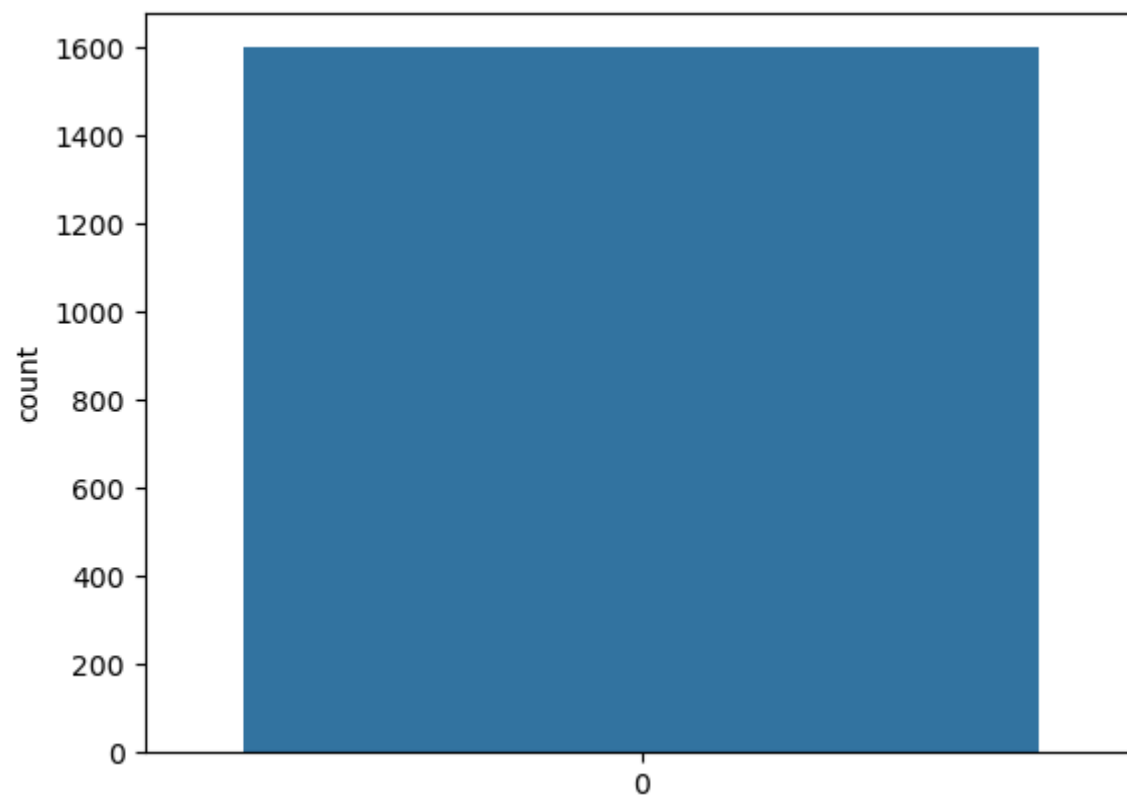
	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
quality											
3	8.360000	0.884500	0.171000	2.635000	0.122500	11.000000	24.900000	0.997464	3.398000	0.570000	9.955000
4	7.779245	0.693962	0.174151	2.694340	0.090679	12.264151	36.245283	0.996542	3.381509	0.596415	10.265094
5	8.167254	0.577041	0.243686	2.528855	0.092736	16.983847	56.513950	0.997104	3.304949	0.620969	9.899706
6	8.347179	0.497484	0.273824	2.477194	0.084956	15.711599	40.869906	0.996615	3.318072	0.675329	10.629519
7	8.872362	0.403920	0.375176	2.720603	0.076588	14.045226	35.020101	0.996104	3.290754	0.741256	11.465913
8	8.566667	0.423333	0.391111	2.577778	0.068444	13.277778	33.444444	0.995212	3.267222	0.767778	12.094444

DATA ANALYSIS USING SEABORN

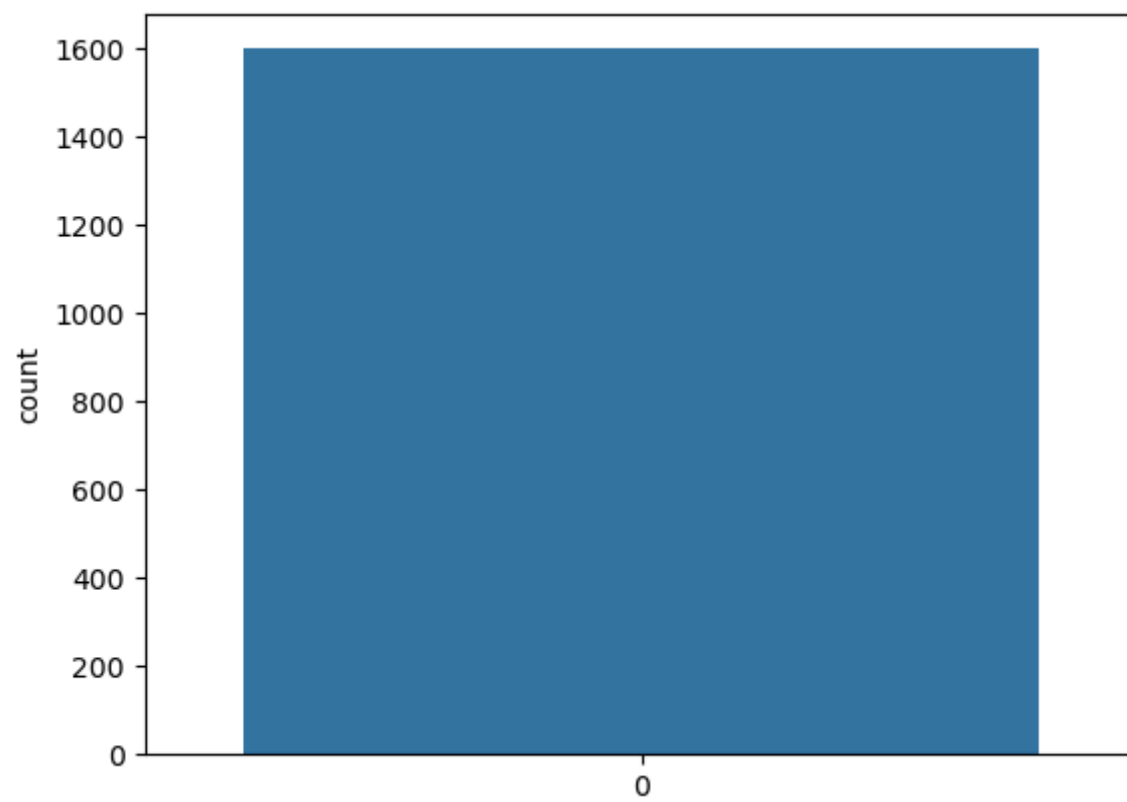
```
In [11]: sns.countplot(wine['quality'])
plt.show()
```



```
In [12]: ▶ sns.countplot(wine['alcohol'])  
plt.show()
```

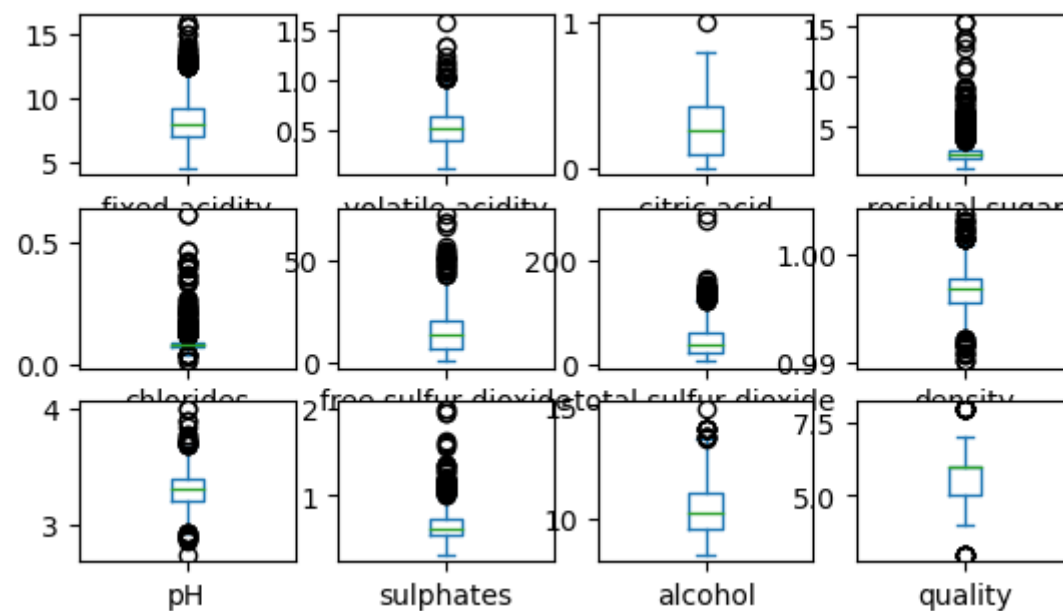


```
In [13]: ▶ sns.countplot(wine['pH'])  
plt.show()
```



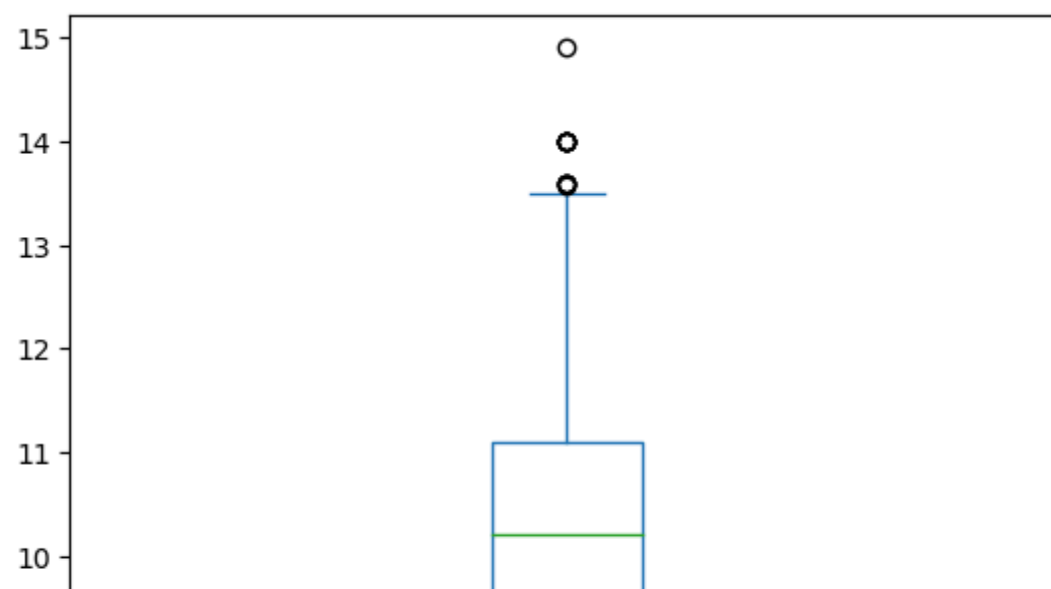
```
In [14]: wine.plot(kind='box',subplots=True,layout=(4,4),sharex=False)
```

```
Out[14]: fixed acidity      Axes(0.125,0.712609;0.168478x0.167391)
volatile acidity  Axes(0.327174,0.712609;0.168478x0.167391)
citric acid       Axes(0.529348,0.712609;0.168478x0.167391)
residual sugar    Axes(0.731522,0.712609;0.168478x0.167391)
chlorides         Axes(0.125,0.511739;0.168478x0.167391)
free sulfur dioxide Axes(0.327174,0.511739;0.168478x0.167391)
total sulfur dioxide Axes(0.529348,0.511739;0.168478x0.167391)
density          Axes(0.731522,0.511739;0.168478x0.167391)
pH               Axes(0.125,0.31087;0.168478x0.167391)
sulphates        Axes(0.327174,0.31087;0.168478x0.167391)
alcohol          Axes(0.529348,0.31087;0.168478x0.167391)
quality          Axes(0.731522,0.31087;0.168478x0.167391)
dtype: object
```



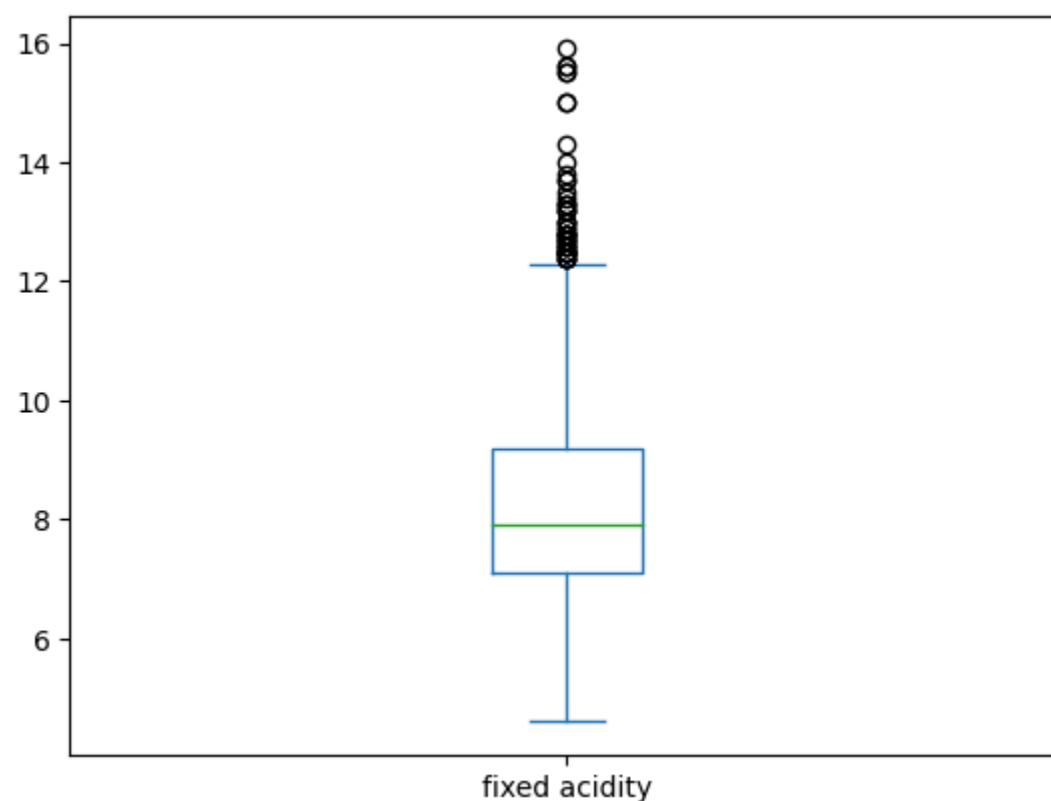
```
In [15]: wine['alcohol'].plot(kind='box')
```

```
Out[15]: <Axes: >
```



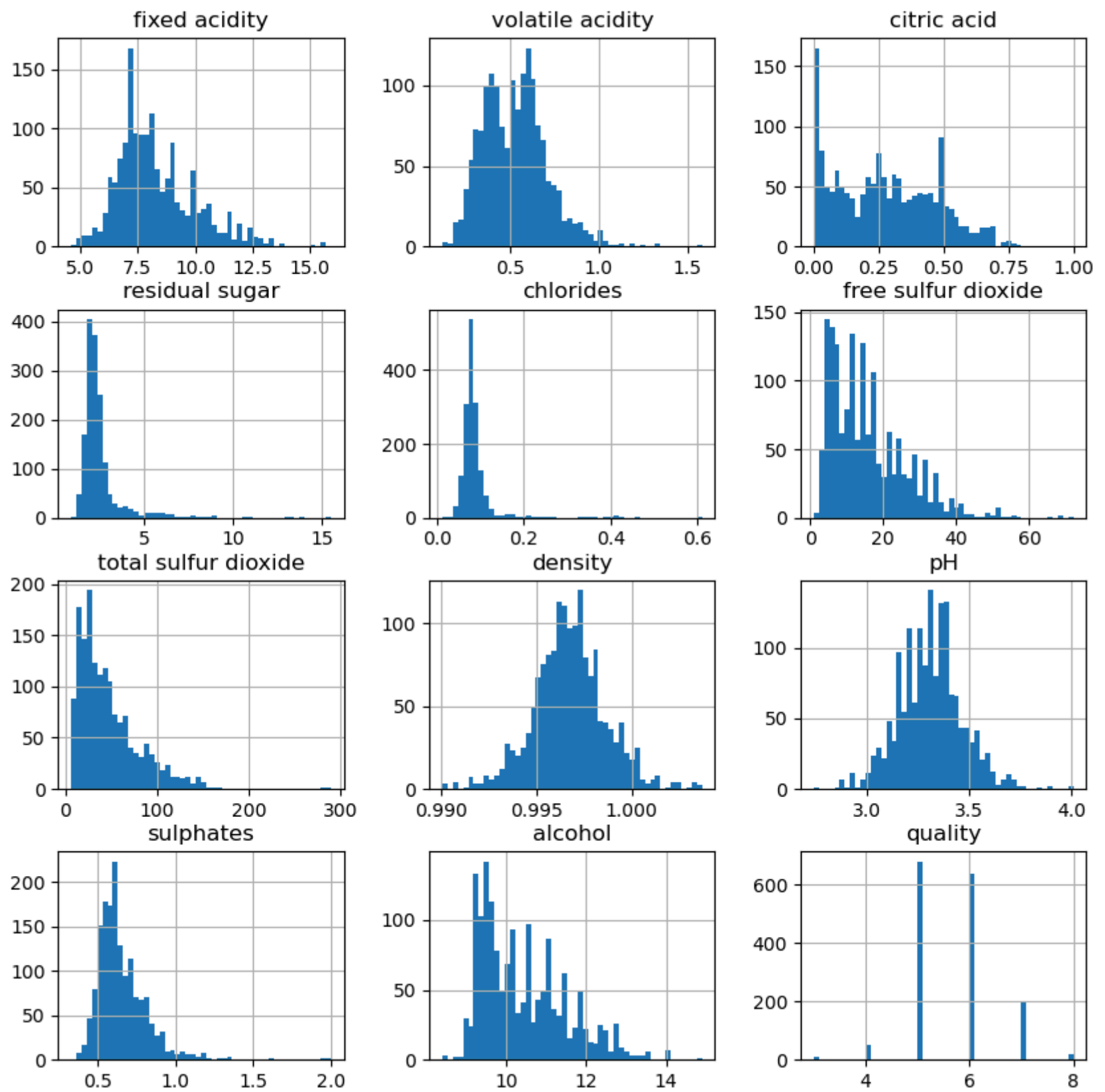
```
In [16]: wine['fixed acidity'].plot(kind='box')
```

```
Out[16]: <Axes: >
```



HISTOGRAM

```
In [17]: wine.hist(figsize=(10,10),bins=50)
plt.show()
```

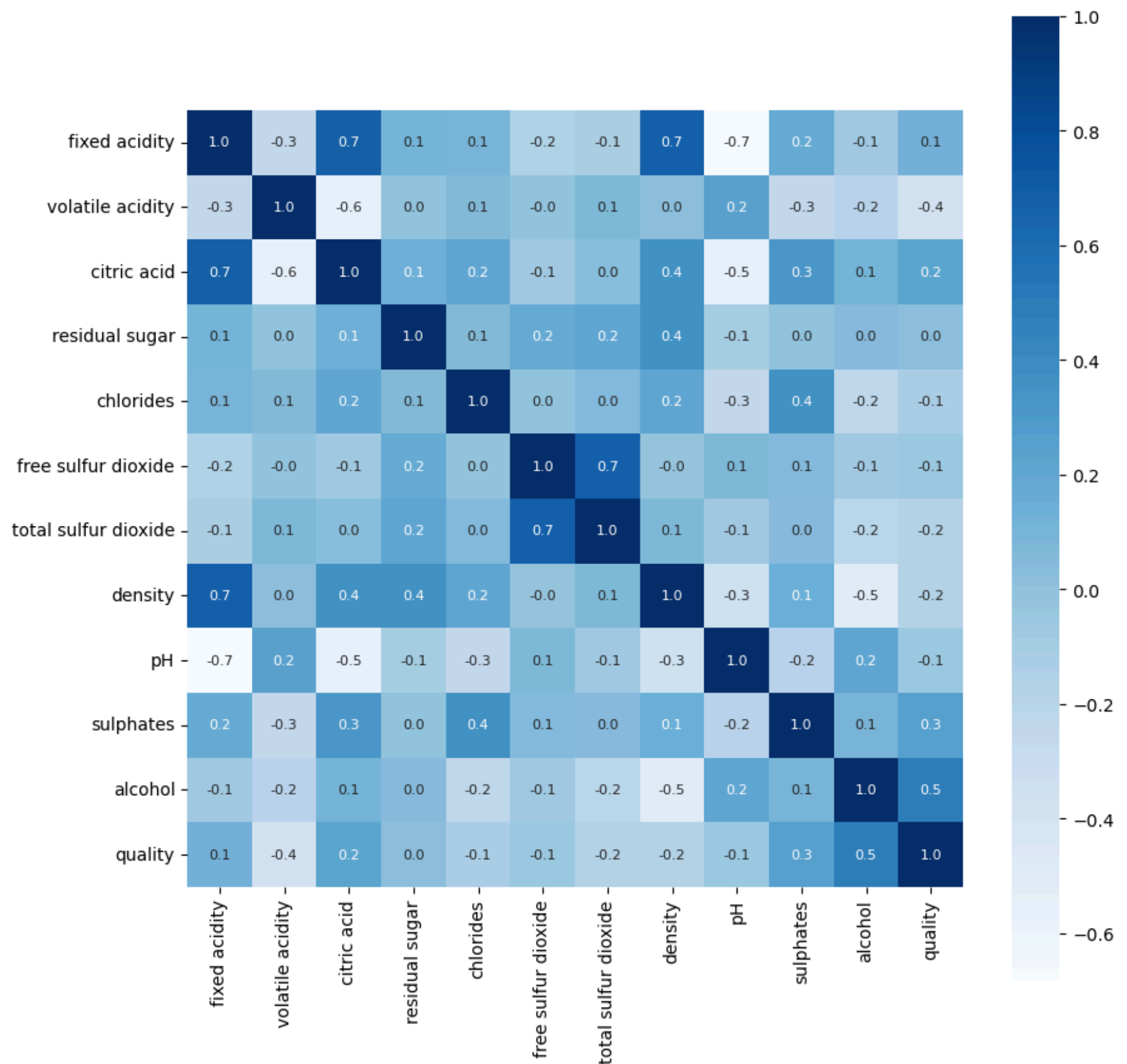


```
In [18]: correlation = wine.corr()
```



```
In [19]: ▶ plt.figure(figsize=(10,10))
sns.heatmap(correlation, cbar=True, square=True, fmt='.1f', annot=True, annot_kws={'size':8}, cmap='Blues')
```

Out[19]: <Axes: >



FEATURE SELECTION

```
In [20]: ▶ wine.sample(5)
```

Out[20]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality
911	9.1	0.28	0.46	9.0	0.114	3.0	9.0	0.99901	3.18	0.60	10.9	6
552	9.5	0.46	0.24	2.7	0.092	14.0	44.0	0.99800	3.12	0.74	10.0	6
343	10.9	0.39	0.47	1.8	0.118	6.0	14.0	0.99820	3.30	0.75	9.8	6
1458	8.4	0.29	0.40	1.7	0.067	8.0	20.0	0.99603	3.39	0.60	10.5	5
1575	7.5	0.52	0.40	2.2	0.060	12.0	20.0	0.99474	3.26	0.64	11.8	6

```
In [21]: ▶ wine['quality'].unique()
```

Out[21]: array([5, 6, 7, 4, 8, 3], dtype=int64)

Condition given : If the wine quality is equal or greater than 7 then it is considered as good wine

```
In [22]: wine['goodquality'] = [1 if x >= 7 else 0 for x in wine['quality']]
wine.sample(5)
```

Out[22]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	goodquality
1165	8.5	0.44	0.50	1.9	0.369	15.0	38.0	0.99634	3.01	1.10	9.4	5	0
262	8.0	0.52	0.03	1.7	0.070	10.0	35.0	0.99575	3.34	0.57	10.0	5	0
746	8.2	0.34	0.38	2.5	0.080	12.0	57.0	0.99780	3.30	0.47	9.0	6	0
113	10.1	0.31	0.44	2.3	0.080	22.0	46.0	0.99880	3.32	0.67	9.7	6	0
6	7.9	0.60	0.06	1.6	0.069	15.0	59.0	0.99640	3.30	0.46	9.4	5	0

```
In [23]: wine['goodquality'] = [1 if x >= 7 else 0 for x in wine['quality']]
wine.sample(25)
```

Out[23]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol	quality	goodquality
773	7.9	0.400	0.29	1.8	0.157	1.0	44.0	0.99730	3.30	0.92	9.5	6	0
770	7.1	0.590	0.02	2.3	0.082	24.0	94.0	0.99744	3.55	0.53	9.7	6	0
932	7.6	0.400	0.29	1.9	0.078	29.0	66.0	0.99710	3.45	0.59	9.5	6	0
1306	9.7	0.690	0.32	2.5	0.088	22.0	91.0	0.99790	3.29	0.62	10.1	5	0
843	7.1	0.685	0.35	2.0	0.088	9.0	92.0	0.99630	3.28	0.62	9.4	5	0
129	6.1	0.380	0.15	1.8	0.072	6.0	19.0	0.99550	3.42	0.57	9.4	5	0
960	8.5	0.470	0.27	1.9	0.058	18.0	38.0	0.99518	3.16	0.85	11.1	6	0
1255	6.1	0.640	0.02	2.4	0.069	26.0	46.0	0.99358	3.47	0.45	11.0	5	0
643	9.5	0.590	0.44	2.3	0.071	21.0	68.0	0.99920	3.46	0.63	9.5	5	0
600	8.2	0.915	0.27	2.1	0.088	7.0	23.0	0.99620	3.26	0.47	10.0	4	0
10	6.7	0.580	0.08	1.8	0.097	15.0	65.0	0.99590	3.28	0.54	9.2	5	0
1171	7.1	0.590	0.00	2.2	0.078	26.0	44.0	0.99522	3.42	0.68	10.8	6	0
584	11.8	0.330	0.49	3.4	0.093	54.0	80.0	1.00020	3.30	0.76	10.7	7	1
612	7.5	0.640	0.00	2.4	0.077	18.0	29.0	0.99650	3.32	0.60	10.0	6	0
1192	7.2	0.250	0.37	2.5	0.063	11.0	41.0	0.99439	3.52	0.80	12.4	7	1
1549	7.4	0.360	0.30	1.8	0.074	17.0	24.0	0.99419	3.24	0.70	11.4	8	1
529	9.9	0.630	0.24	2.4	0.077	6.0	33.0	0.99740	3.09	0.57	9.4	5	0
1435	10.2	0.540	0.37	15.4	0.214	55.0	95.0	1.00369	3.18	0.77	9.0	6	0
409	12.5	0.460	0.49	4.5	0.070	26.0	49.0	0.99810	3.05	0.57	9.6	4	0
1493	7.7	0.540	0.26	1.9	0.089	23.0	147.0	0.99636	3.26	0.59	9.7	5	0
1485	7.0	0.655	0.16	2.1	0.074	8.0	25.0	0.99606	3.37	0.55	9.7	5	0
412	7.1	0.735	0.16	1.9	0.100	15.0	77.0	0.99660	3.27	0.64	9.3	5	0
16	8.5	0.280	0.56	1.8	0.092	35.0	103.0	0.99690	3.30	0.75	10.5	7	1
202	6.8	0.500	0.11	1.5	0.075	16.0	49.0	0.99545	3.36	0.79	9.5	5	0
813	6.9	0.390	0.24	2.1	0.102	4.0	7.0	0.99462	3.44	0.58	11.4	4	0

Filtering the quality wine out of the dataset

```
In [24]: wine['goodquality'].value_counts()
```

```
Out[24]: 0    1382
         1     217
         Name: goodquality, dtype: int64
```

Hence,
 "0" - contains below of quality
 "1" - contains good quality

Separating depedent and indepedent variables

```
In [25]: X = wine.drop(['quality', 'goodquality'], axis = 1)
         Y = wine['goodquality']
```



```
In [26]: X
```

Out[26]:

	fixed acidity	volatile acidity	citric acid	residual sugar	chlorides	free sulfur dioxide	total sulfur dioxide	density	pH	sulphates	alcohol
0	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4
1	7.8	0.880	0.00	2.6	0.098	25.0	67.0	0.99680	3.20	0.68	9.8
2	7.8	0.760	0.04	2.3	0.092	15.0	54.0	0.99700	3.26	0.65	9.8
3	11.2	0.280	0.56	1.9	0.075	17.0	60.0	0.99800	3.16	0.58	9.8
4	7.4	0.700	0.00	1.9	0.076	11.0	34.0	0.99780	3.51	0.56	9.4
...	...	...	...	...	...	...	...	...	...	...	...
1594	6.2	0.600	0.08	2.0	0.090	32.0	44.0	0.99490	3.45	0.58	10.5
1595	5.9	0.550	0.10	2.2	0.062	39.0	51.0	0.99512	3.52	0.76	11.2
1596	6.3	0.510	0.13	2.3	0.076	29.0	40.0	0.99574	3.42	0.75	11.0
1597	5.9	0.645	0.12	2.0	0.075	32.0	44.0	0.99547	3.57	0.71	10.2
1598	6.0	0.310	0.47	3.6	0.067	18.0	42.0	0.99549	3.39	0.66	11.0

1599 rows × 11 columns

```
In [27]: Y
```

Out[27]:

```
0      0
1      0
2      0
3      0
4      0
..
1594    0
1595    0
1596    0
1597    0
1598    0
Name: goodquality, Length: 1599, dtype: int64
```

```
In [28]: Y.sample(20)
```

Out[28]:

```
1443    0
1208    1
911     0
469     0
410     0
77      0
1       0
1087    0
815     0
579     0
772     0
1054    0
594     0
314     0
1122    0
797     1
1207    0
282     0
565     0
1080    0
Name: goodquality, dtype: int64
```

FEATURE IMPORTANCE

```
In [29]: from sklearn.ensemble import ExtraTreesClassifier
classifiern = ExtraTreesClassifier()
classifiern.fit(X,Y)
score = classifiern.feature_importances_
print(score)

[0.07600466 0.097593  0.10082118 0.07239497 0.06787239 0.07014282
 0.08064517 0.08995089 0.06717917 0.10978319 0.16761256]
```

SPLITTING DATASET

```
In [30]: from sklearn.model_selection import train_test_split
X_train, X_test, Y_train, Y_test = train_test_split(X,Y,test_size=0.3,random_state=7)
```

RESULT

```
In [53]: >> model_res=pd.DataFrame(columns=['Model', ' Accuracy Score'])
```

APPYLING DIFFERENT ALGORITHMS FOR CHECKING ACCURACY SCORE

1) LOGISTIC REGRESSION

```
In [54]: >> from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score,confusion_matrix
# accuracy_score(Y_test,Y_pred)
model_res.loc[len(model_res)] = ['Logistic Regression', accuracy_score(Y_test,y_pred)]
model_res
```

Out[54]:

	Model	Accuracy Score
0	Logistic Regression	0.86875

2) KNN (K Nearest Neighbour)

```
In [55]: >> from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score
model_res.loc[len(model_res)] = ['K Neighbors Classifier', accuracy_score(Y_test,y_pred)]
model_res
```

Out[55]:

	Model	Accuracy Score
0	Logistic Regression	0.868750
1	K Neighbors Classifier	0.808333

3) SVC (Support Vector Classifier)

```
In [56]: >> from sklearn.svm import SVC
model = SVC()
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score
print("Accuracy Score:",accuracy_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['SVC', accuracy_score(Y_test,y_pred)]
model_res
```

Accuracy Score: 0.86875

Out[56]:

	Model	Accuracy Score
0	Logistic Regression	0.868750
1	K Neighbors Classifier	0.808333
2	SVC	0.868750

4) Decision Tree

```
In [57]: ► from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(criterion='entropy',random_state=7)
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import accuracy_score
print("Accuracy Score:",accuracy_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['Decision Tree Classifier', accuracy_score(Y_test,y_pred)]
model_res
```

Accuracy Score: 0.7958333333333333

Out[57]:

	Model	Accuracy Score
0	Logistic Regression	0.868750
1	K Neighbors Classifier	0.808333
2	SVC	0.868750
3	Decision Tree Classifier	0.795833

5) GaussianNB (Gaussian Naive Bayes)

```
In [58]: ► from sklearn.naive_bayes import GaussianNB
model3 = GaussianNB()
model3.fit(X_train,Y_train)
y_pred = model3.predict(X_test)

from sklearn.metrics import accuracy_score
print("Accuracy Score:",accuracy_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['GaussianNB', accuracy_score(Y_test,y_pred)]
model_res
```

Accuracy Score: 0.8479166666666667

Out[58]:

	Model	Accuracy Score
0	Logistic Regression	0.868750
1	K Neighbors Classifier	0.808333
2	SVC	0.868750
3	Decision Tree Classifier	0.795833
4	GaussianNB	0.847917

6) Random Forest

```
In [59]: ► from sklearn.ensemble import RandomForestClassifier
model2 = RandomForestClassifier(random_state=1)
model2.fit(X_train, Y_train)
y_pred = model2.predict(X_test)

from sklearn.metrics import accuracy_score
print("Accuracy Score:",accuracy_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['Random Forest Classifier', accuracy_score(Y_test,y_pred)]
model_res
```

Accuracy Score: 0.8354166666666667

Out[59]:

	Model	Accuracy Score
0	Logistic Regression	0.868750
1	K Neighbors Classifier	0.808333
2	SVC	0.868750
3	Decision Tree Classifier	0.795833
4	GaussianNB	0.847917
5	Random Forest Classifier	0.835417

```
In [38]: ► pip install xgboost
```

Requirement already satisfied: xgboost in c:\users\adISR\anaconda3\lib\site-packages (2.0.3)
Requirement already satisfied: numpy in c:\users\adISR\anaconda3\lib\site-packages (from xgboost) (1.24.3)
Requirement already satisfied: scipy in c:\users\adISR\anaconda3\lib\site-packages (from xgboost) (1.10.1)
Note: you may need to restart the kernel to use updated packages.

7) XGBoost (Extreme Gradient Boosting)

```
In [60]: ▶ import xgboost as xgb
model5 = xgb.XGBClassifier(random_state=1)
model5.fit(X_train, Y_train)
y_pred = model5.predict(X_test)

from sklearn.metrics import accuracy_score
print("Accuracy Score:",accuracy_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['XGBClassifier', accuracy_score(Y_test,y_pred)]
model_res
```

Accuracy Score: 0.8229166666666666

Out[60]:

	Model	Accuracy Score
0	Logistic Regression	0.868750
1	K Neighbors Classifier	0.808333
2	SVC	0.868750
3	Decision Tree Classifier	0.795833
4	GaussianNB	0.847917
5	Random Forest Classifier	0.835417
6	XGBClassifier	0.822917

RESULT AS A WHOLE :

```
In [62]: ▶ model_res = model_res.sort_values(by=' Accuracy Score', ascending=False)
model_res
```

Out[62]:

	Model	Accuracy Score
0	Logistic Regression	0.868750
2	SVC	0.868750
4	GaussianNB	0.847917
5	Random Forest Classifier	0.835417
6	XGBClassifier	0.822917
1	K Neighbors Classifier	0.808333
3	Decision Tree Classifier	0.795833

```
In [64]: ▶ model_res = model_res.sort_values(by=' Accuracy Score', ascending=True)
model_res
```

Out[64]:

	Model	Accuracy Score
3	Decision Tree Classifier	0.795833
1	K Neighbors Classifier	0.808333
6	XGBClassifier	0.822917
5	Random Forest Classifier	0.835417
4	GaussianNB	0.847917
0	Logistic Regression	0.868750
2	SVC	0.868750

REPORT FOR ACCURACY SCORE :

APPLYING DIFFERENT ALOGORITHMS TO CHECK PRECISION SCORE

```
In [118]: ▶ model_res=pd.DataFrame(columns=['Model', 'Precision Score'])
```

1. LOGISTIC REGRESSION

```
In [119]: from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import precision_score,confusion_matrix
model_res.loc[len(model_res)] = ['Logistic Regression', precision_score(Y_test,y_pred)]
model_res
```

Out[119]:

	Model	Precision Score
0	Logistic Regression	0.0

2. KNN

```
In [120]: from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import precision_score
model_res.loc[len(model_res)] = ['K Neighbors Classifier', precision_score(Y_test,y_pred)]
model_res
```

Out[120]:

	Model	Precision Score
0	Logistic Regression	0.000000
1	K Neighbors Classifier	0.085714

3. SVC

```
In [121]: from sklearn.svm import SVC
model = SVC()
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import precision_score
print("Precision Score:",precision_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['SVC', precision_score(Y_test,y_pred)]
model_res
```

Precision Score: 0.0

Out[121]:

	Model	Precision Score
0	Logistic Regression	0.000000
1	K Neighbors Classifier	0.085714
2	SVC	0.000000

4. DECISION TREE

```
In [122]: from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(criterion='entropy',random_state=7)
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import precision_score
print("Precision Score:",precision_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['Decision Tree Classifier', precision_score(Y_test,y_pred)]
model_res
```

Precision Score: 0.18181818181818182

Out[122]:

	Model	Precision Score
0	Logistic Regression	0.000000
1	K Neighbors Classifier	0.085714
2	SVC	0.000000
3	Decision Tree Classifier	0.181818

5. GUASSIAN NB

```
In [123]: from sklearn.naive_bayes import GaussianNB
model3 = GaussianNB()
model3.fit(X_train,Y_train)
y_pred = model3.predict(X_test)

from sklearn.metrics import precision_score
print("Precision Score:",precision_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['GaussianNB', precision_score(Y_test,y_pred)]
model_res
```

Precision Score: 0.1875

Out[123]:

	Model	Precision Score
0	Logistic Regression	0.000000
1	K Neighbors Classifier	0.085714
2	SVC	0.000000
3	Decision Tree Classifier	0.181818
4	GaussianNB	0.187500

6. RANDOM FOREST

```
In [124]: from sklearn.ensemble import RandomForestClassifier
model2 = RandomForestClassifier(random_state=1)
model2.fit(X_train, Y_train)
y_pred = model2.predict(X_test)

from sklearn.metrics import precision_score
print("Precision Score:",precision_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['Random Forest Classifier', precision_score(Y_test,y_pred)]
model_res
```

Precision Score: 0.0

Out[124]:

	Model	Precision Score
0	Logistic Regression	0.000000
1	K Neighbors Classifier	0.085714
2	SVC	0.000000
3	Decision Tree Classifier	0.181818
4	GaussianNB	0.187500
5	Random Forest Classifier	0.000000

7. XGBOOST

```
In [125]: import xgboost as xgb
model5 = xgb.XGBClassifier(random_state=1)
model5.fit(X_train, Y_train)
y_pred = model5.predict(X_test)

from sklearn.metrics import precision_score
print("Precision Score:",precision_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['XGBClassifier', precision_score(Y_test,y_pred)]
model_res
```

Precision Score: 0.07692307692307693

Out[125]:

	Model	Precision Score
0	Logistic Regression	0.000000
1	K Neighbors Classifier	0.085714
2	SVC	0.000000
3	Decision Tree Classifier	0.181818
4	GaussianNB	0.187500
5	Random Forest Classifier	0.000000
6	XGBClassifier	0.076923

REPORT FOR PRECISION SCORE

--	--

APPLYING DIFFERENT ALGORITHMS FOR RECALL SCORE

```
In [131]: ▶ model_res=pd.DataFrame(columns=['Model', 'Recall Score'])
```

```

In [132]: ► from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import recall_score,confusion_matrix
model_res.loc[len(model_res)] = ['Logistic Regression', recall_score(Y_test,y_pred)]
model_res

from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import recall_score
model_res.loc[len(model_res)] = ['K Neighbors Classifier', recall_score(Y_test,y_pred)]
model_res

from sklearn.svm import SVC
model = SVC()
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import recall_score
print("Recall Score:",recall_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['SVC', recall_score(Y_test,y_pred)]
model_res

from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(criterion='entropy',random_state=7)
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import recall_score
print("Recall Score:",recall_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['Decision Tree Classifier', recall_score(Y_test,y_pred)]
model_res

from sklearn.naive_bayes import GaussianNB
model3 = GaussianNB()
model3.fit(X_train,Y_train)
y_pred = model3.predict(X_test)

from sklearn.metrics import recall_score
print("Recall Score:",recall_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['GaussianNB', recall_score(Y_test,y_pred)]
model_res

from sklearn.ensemble import RandomForestClassifier
model2 = RandomForestClassifier(random_state=1)
model2.fit(X_train, Y_train)
y_pred = model2.predict(X_test)

from sklearn.metrics import recall_score
print("Recall Score:",recall_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['Random Forest Classifier', recall_score(Y_test,y_pred)]
model_res

import xgboost as xgb
model5 = xgb.XGBClassifier(random_state=1)
model5.fit(X_train, Y_train)
y_pred = model5.predict(X_test)

from sklearn.metrics import recall_score
print("Recall Score:",recall_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['XGBClassifier', recall_score(Y_test,y_pred)]
model_res

```

```

Recall Score: 0.0
Recall Score: 0.15873015873015872
Recall Score: 0.047619047619047616
Recall Score: 0.0
Recall Score: 0.031746031746031744

```


Out[132]:

	Model	Recall Score
0	Logistic Regression	0.000000
1	K Neighbors Classifier	0.047619
2	SVC	0.000000
3	Decision Tree Classifier	0.158730
4	GaussianNB	0.047619
5	Random Forest Classifier	0.000000
6	XGBClassifier	0.031746

REPORT FOR RECALL SCORE

In []:

▶

APPLYING DIFFERENT ALGORITHMS FOR f1 SCORE

```

In [133]: model_res=pd.DataFrame(columns=['Model', 'f1 Score'])

from sklearn.linear_model import LogisticRegression
model = LogisticRegression()
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import f1_score,confusion_matrix
model_res.loc[len(model_res)] = ['Logistic Regression', f1_score(Y_test,y_pred)]
model_res

from sklearn.neighbors import KNeighborsClassifier
model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import f1_score
model_res.loc[len(model_res)] = ['K Neighbors Classifier', f1_score(Y_test,y_pred)]
model_res

from sklearn.svm import SVC
model = SVC()
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import f1_score
print("f1 Score:",f1_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['SVC', f1_score(Y_test,y_pred)]
model_res

from sklearn.tree import DecisionTreeClassifier
model = DecisionTreeClassifier(criterion='entropy',random_state=7)
model.fit(X_train,Y_train)
y_pred = model.predict(X_test)

from sklearn.metrics import f1_score
print("f1 Score:",f1_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['Decision Tree Classifier', f1_score(Y_test,y_pred)]
model_res

from sklearn.naive_bayes import GaussianNB
model3 = GaussianNB()
model3.fit(X_train,Y_train)
y_pred = model3.predict(X_test)

from sklearn.metrics import f1_score
print("f1 Score:",f1_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['GaussianNB', f1_score(Y_test,y_pred)]
model_res

from sklearn.ensemble import RandomForestClassifier
model2 = RandomForestClassifier(random_state=1)
model2.fit(X_train, Y_train)
y_pred = model2.predict(X_test)

from sklearn.metrics import f1_score
print("f1 Score:",f1_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['Random Forest Classifier', f1_score(Y_test,y_pred)]
model_res

import xgboost as xgb
model5 = xgb.XGBClassifier(random_state=1)
model5.fit(X_train, Y_train)
y_pred = model5.predict(X_test)

from sklearn.metrics import f1_score
print("f1 Score:",f1_score(Y_test,y_pred))
model_res.loc[len(model_res)] = ['XGBClassifier', f1_score(Y_test,y_pred)]
model_res

```

```

f1 Score: 0.0
f1 Score: 0.1694915254237288
f1 Score: 0.0759493670886076
f1 Score: 0.0
f1 Score: 0.0449438202247191

```

Out[133]:

	Model	f1 Score
0	Logistic Regression	0.000000
1	K Neighbors Classifier	0.061224
2	SVC	0.000000
3	Decision Tree Classifier	0.169492
4	GaussianNB	0.075949
5	Random Forest Classifier	0.000000
6	XGBClassifier	0.044944

REPORT FOR f1 SCORE

In []: ▶

APPLYING DIFFERENT ALGORITHMS FOR ROC

In []: ▶